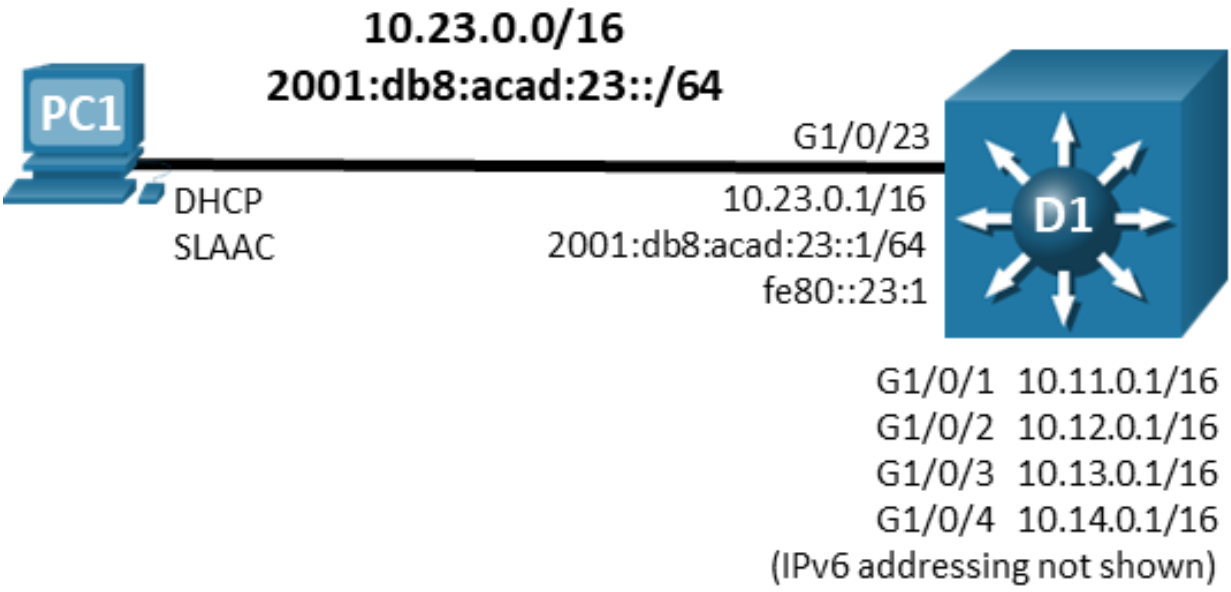
Lab - Use NETCONF to Access an IOS XE Layer 3 Switch

Topology



Addressing Table

Device	Interface	IP Address	Subnet Mask
D1	G1/0/23	10.23.0.1	255.255.0.0
PC	NIC	DHCP	DHCP

Objectives

Part 1: Configure an IOS XE Device for SSH Access

Part 2: Configure and Verify NETCONF on D1

Part 3: Using NETCONF Interactive Mode to Retrieve Interface Information

Part 4: Using NETCONF Interactive Mode to Retrieve Interface-State Information

Part 5: Using NETCONF with Python

Part 6: Using Python and NETCONF to Retrieve Device Configuration

Part 7: Using Python and NETCONF to Retrieve Interface Information

Part 8: Using Python and NETCONF to Configure a Device

Background / Scenario

NETCONF (Network Configuration Protocol) is a network management protocol defined in RFC 6241. It is designed for **programmatic configuration**, not for manually typing CLI commands. NETCONF works closely

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

with **YANG**, a data-modeling language used by many vendors—including Cisco IOS XE—to describe configuration and operational data in a consistent, structured way.

When you interact with an IOS XE device using NETCONF:

- The device sends and receives data in **XML format**
- The structure of that XML is defined by **YANG models**
- Every configuration change is treated as a **transaction**, meaning the device applies *all* of the change or *none* of it

In this lab, you will:

1. Configure and verify NETCONF is running on an IOS XE device
2. Connect to NETCONF using **ncclient**, a Python library for NETCONF
3. Retrieve and modify device configuration using **NETCONF interactive mode and YANG-based filters**
4. Retrieve and modify device configuration using **Python, ncclient and YANG-based filters**

If you have already completed the RESTCONF lab, you will notice that NETCONF returns similar information, but instead of sending REST URLs and receiving JSON or XML, NETCONF communicates using structured XML RPC messages. Both NETCONF and RESTCONF rely on the **same YANG models**, which is why the data you retrieve looks familiar across both labs.

Required Resources

- 1 Switch (Cisco 9300L with Cisco IOS XE Release 17.12.5 universal image or comparable)
- 1 PCs (Choice of operating system with terminal emulation program installed and Postman installed)
- Console cables to configure the Cisco IOS devices via the console ports
- Ethernet cables as shown in the topology

Part 1: Configure an IOS XE Device for SSH Access

Before a device can accept NETCONF connections, it must be reachable over the network and properly configured for SSH access. NETCONF runs over SSH on port 830, so the IOS XE device must have a hostname, domain name, user credentials, RSA keys, and an interface with IP connectivity. In this first section, you will configure D1 with basic settings, IP addressing, and SSH so that it can receive NETCONF requests later in the lab.

Step 1: Configure D1.

Console into D1 and paste the following configuration into the CLI to configure basic settings including enabling SSH.

```
hostname D1
no ip domain lookup
enable secret class
ipv6 unicast-routing
interface range g1/0/1-24
 shutdown
 exit
line con 0
 logg sync
 exec-time 0 0
 exit
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
interface g1/0/23
 no switchport
 description Link to PC
 ip add 10.23.0.1 255.255.0.0
 ipv6 add 2001:db8:acad:23::1/64
 ipv6 add fe80::23:1 link-local
 no shutdown
 exit
interface g1/0/1
 no switchport
 description Link to Sales network
 ip add 10.11.0.1 255.255.0.0
 ipv6 add 2001:db8:acad:11::1/64
 no shutdown
 exit
interface g1/0/2
 no switchport
 description Link to Engineering network
 ip add 10.12.0.1 255.255.0.0
 ipv6 add 2001:db8:acad:12::1/64
 no shutdown
 exit
interface g1/0/3
 no switchport
 description Link to HR network
 ip add 10.13.0.1 255.255.0.0
 ipv6 add 2001:db8:acad:13::1/64
 no shutdown
 exit
interface g1/0/4
 no switchport
 description Link to DC network
 ip add 10.14.0.1 255.255.0.0
 ipv6 add 2001:db8:acad:14::1/64
 no shutdown
 exit
ip domain name example.netacad.com
ip dhcp excluded-address 10.23.0.1 10.23.0.9
ip dhcp pool LAN
 network 10.23.0.0 255.255.0.0
 default-router 10.23.0.1
 domain-name example.netacad.com
 exit
username cisco privilege 15 password 0 cisco123!
crypto key generate rsa general-keys modulus 2048
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
line vty 0 15
 exec-timeout 0 0
 logging synchronous
 login local
 transport input ssh
 exit
line con 0
 password cisco
 login
 exit
end
copy run start
```

Step 2: Verify that PC1 can ping the default gateway.

a. On PC1, open a command prompt and enter the command to verify that the PC has received an IP address in the **10.23.0.0/16** network and that the default gateway is **10.23.0.1**.

b. Test connectivity to D1 using the command **ping 10.23.0.1**

c. If the ping fails, troubleshoot by checking interface status, addressing, cables, or the DHCP configuration.

Step 3: Establish an SSH Connection to D1.

a. From PC1, SSH into the IOS XE device: **ssh cisco@10.23.0.1**

b. When prompted, enter the password: **cisco123!**

c. When you can log in successfully, proceed to Part 2 to verify NETCONF services.

If SSH fails, confirm that:

- The RSA keys were generated (**crypto key generate rsa**)
- VTY lines are configured for SSH (**transport input ssh**)
- You are using the correct **username** and **password**
- The D1 interface, 10.23.0.1, is reachable

Part 2: Configure and Verify NETCONF on D1

NETCONF uses several background processes on an IOS XE device. These processes are part of the *YANG management subsystem*, which handles NETCONF and RESTCONF operations behind the scenes. To use NETCONF, the key process you must verify is **ncsshd**, the NETCONF-over-SSH daemon. However, other services must also be running.

In this section, you will confirm whether NETCONF is already running and enable it if necessary.

Step 1: Verify if NETCONF is Running on D1.

a. NETCONF may already be enabled (for example, if another user enabled it earlier, or if the IOS XE version enables it by default).

Use the following command to view the NETCONF/YANG processes:

```
D1# show platform software yang-management process
confd            : Not Running
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
nesd              : Not Running
syncfd            : Not Running
ncsshd            : Not Running
dmiauthd          : Not Running
nginx             : Running
ndbmand           : Not Running
pubd              : Not Running
gnmib             : Not Running
```

The **ncsshd** service is the NETCONF-over-SSH daemon on IOS XE. It listens on TCP port **830** and is the service that **ncclient** connects to when sending NETCONF RPC messages. Although several YANG-management processes run in the background (such as **confd**, **nesd**, **syncfd**, and **dmiauthd**), **ncclient** only interacts directly with **ncsshd**.

The other processes work behind the scenes to process YANG models, manage datastores, synchronize configuration, and handle authentication.

Explanation of other key processes that should be running if NETCONF was enabled:

ncsshd: NETCONF SSH Daemon

- Listens on TCP port **830** and accepts NETCONF RPC messages; **required for NETCONF**.

confd: YANG/NETCONF Core Engine

- Handles YANG models and processes NETCONF/RESTCONF operations; **required for NETCONF**.

nesd: Datastore Engine

- Manages the YANG configuration and operational datastores; **required for NETCONF**.

syncfd: Datastore Synchronization

- Keeps the CLI configuration and YANG datastore consistent; **required for NETCONF**.

dmiauthd: Authentication Service

- Authenticates NETCONF and RESTCONF sessions; **required for NETCONF**.

nginx: RESTCONF Web Server

- Provides RESTCONF HTTPS access; **not required for NETCONF** (but often running).

gnmib: gNMI/gRPC Telemetry Service

- Supports gNMI/gRPC-based model-driven telemetry; **not required for NETCONF** but may run on devices that support streaming telemetry.

Enabling NETCONF is required before you can send NETCONF RPC messages or use **ncclient**. If these services are not running, all NETCONF connections will fail even if normal SSH on port 22 is working.

Step 2: Enable NETCONF if required on D1.

If the output in Step 2 shows that **ncsshd** and the other YANG-management processes are **Not Running**, you must enable NETCONF

a. Enter global configuration mode and enable NETCONF/YANG support:

```
D1# config t
D1(config)# netconf-yang
```

When you enable **netconf-yang**, the IOS XE device will display several system messages confirming that the NETCONF/YANG subsystem is starting.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

The key message is **%PSD_MOD-5-DMI_NOTIFY_NETCONF_START**, which indicates that the NETCONF server (including the **ncsshd** daemon) is being activated.

You may also see messages about synchronizing the running configuration to the YANG datastore (**SYNC_NEEDED**, **SYNC_START**, **SYNC_COMPLETE**), which must occur before NETCONF can accurately return configuration data.

IOS XE may additionally generate **PKI messages** as it creates a self-signed certificate for RESTCONF; this is normal and not required for NETCONF.

b. You can verify that **ncsshd** and the other required YANG-management services are running:

```
D1# show platform software yang-management process
confd            : Running
nesd             : Running
syncfd           : Running
ncsshd           : Running
dmiauthd         : Running
nginx            : Running
ndbmand          : Running
pubd             : Running
gnmib            : Not Running
```

The **ncsshd : Running** confirms that NETCONF is active and ready to accept NETCONF connections from **ncclient** on TCP port 830.

It may take several minutes to sync.

```
*Dec  1 10:52:50.421: %DMI-5-SYNC_COMPLETE: Switch 1 R0/0: dmiauthd: The running
configuration has been synchronized to the NETCONF running data store.
```

Part 3: Using NETCONF Interactive Mode to Retrieve Interface Information

NETCONF supports a special *interactive mode* that lets you communicate directly with the device's NETCONF server (D1) over SSH. In this mode, you send raw XML messages (called NETCONF RPCs, or Remote Procedure Calls) and view the XML replies that the device returns.

NETCONF RPC (Remote Procedure Call)

A NETCONF RPC is an XML message, wrapped in the **<rpc>** element, that the client sends to a device to request an operation such as retrieving configuration data or modifying settings. The device processes this request and returns a corresponding **<rpc-reply>** message, which is another XML response containing either the requested data or the result of the operation. In NETCONF, every **<rpc>** sent by the client always generates a matching **<rpc-reply>** from the device.

XML (Extensible Markup Language)

NETCONF exchanges all data using XML, a structured, hierarchical text format used to represent and transport data. XML is similar to HTML but is used strictly for data, not for displaying information.

XML uses descriptive tags (such as **<interface>** or **<name>**) to describe the meaning of the data. Every tag must have a matching closing tag (for example, **</interface>**), or it may be written in a self-closing form, such as **<interfaces … />**, which represents an empty element. NETCONF uses XML because it is predictable, structured, and easy for both humans and automation tools to parse.

In this lab, every NETCONF message you send or receive (including **<hello>**, **<rpc>**, **<rpc-reply>**, **<get>**, and **<get-config>**) is formatted in XML.

YANG (Yet Another Next Generation)

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

NETCONF was designed to work with data models, and today those models are almost always written in **YANG (Yet Another Next Generation)**, the industry-standard modeling language for modern network devices.

When you retrieve configuration or operational data using **<get>** or **<get-config>**, or when you modify configuration using **<edit-config>**, the data you send and receive is defined by YANG modules (such as **ietf-interfaces**, **ietf-ip**, or **Cisco IOS-XE native models**).

A **YANG model** is a structured blueprint that defines how data on a network device is organized and accessed: what information exists, its format, and relationships. Using YANG models provides a consistent, standardized way to retrieve and configure data across different devices, vendors, and APIs, making automation more reliable and predictable.

There are two types of YANG models:

IETF models: These models start with **ietf-** (such as **ietf-interfaces**) and are standards-based, defined by the Internet Engineering Task Force. Other examples include: **ietf-interfaces**, **ietf-ip**, and **ietf-routing**. These models describe common, vendor-neutral data like interfaces or routing.

Cisco native models: These models start with **Cisco-IOS-** and are Cisco-specific extensions that include features unique to IOS XE. Cisco IOS XE examples include: **Cisco-IOS-XE-vlan-oper** and **Cisco-IOS-XE-native**. These models describe things like VLANs, EEM, QoS, and other platform-specific capabilities.

Note: You can explore any model and its tree structure at yangcatalog.org.

Advantages of using YANG models:

- Produce the **same data structure and field names** across different devices and vendors.
- Make **automation and scripting easier**, since tools can expect consistent output.
- Allow **data to be parsed and analyzed** programmatically for reporting and monitoring.
- Enable **interoperability** between management tools, controllers, and APIs (NETCONF, RESTCONF).

In this part of the lab, you will connect to D1's NETCONF service on TCP port 830, exchange **<hello>** messages, and manually send NETCONF RPC operations to retrieve configuration and operational data defined by YANG models.

Note: NETCONF elements such as **<hello>**, **<rpc>**, **<rpc-reply>**, **<get>**, and **<get-config>** are written with angle brackets because these names refer to the actual XML elements defined in the NETCONF standard (RFC 6241).

Step 1: Access the NETCONF process through an SSH terminal.

a. From PC1, start a NETCONF session by connecting to D1 over SSH on TCP port 830, using the NETCONF subsystem.

Enter the following command followed by the password:

```
C:\Users\Student> ssh -oHostKeyAlgorithms=+ssh-dss cisco@10.23.0.1 -p 830 -s netconf
```

Note: The first time you SSH into the Netconf subsystem, you might be prompted to accept continuing, as shown below. Enter **yes** to proceed and then enter the password for user **cisco**.

```
The authenticity of host '[10.23.0.1]:830 ([10.23.0.1]:830)' can't be established.
RSA key fingerprint is SHA256:slOtq5NB2p3qaBefnk+rCu8yVmI307AloN+WbaQnia4.
This key is not known by any other names.
Are you sure you want to continue connecting yes/no/[fingerprint])? yes
cisco@10.23.0.1's password: cisco123!
```

Here is an explanation of each part of the SSH command**:**

- **ssh**: Starts an SSH connection from the PC to the IOS XE device.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

- **-oHostKeyAlgorithms=+ssh-dss**: Tells SSH to allow the older **ssh-dss** host key algorithm. Some IOS XE devices use DSS keys for NETCONF on port 830, so this option ensures compatibility.
- **cisco@10.23.0.1**: Connects using username **cisco** to device at IP **10.23.0.1**.
- **-p 830**: Use **TCP port 830**, the standard NETCONF-over-SSH port (different from normal SSH on port 22).
- **-s netconf**: Specifies the **NETCONF subsystem**. This tells the router: *"Start a NETCONF session, not a normal SSH shell."*

b. If the SSH connection is successful, D1 immediately sends a NETCONF server **<hello>** message. This is the first message in every NETCONF session. It is **not an RPC request**. It is simply the server (D1) introducing itself to the client (PC).

The server **<hello>** message includes:

- **XML header**: identifies the message as XML

```
<?xml version="1.0" encoding="UTF-8"?>
```

- **NETCONF namespace**: the schema used for NETCONF messages

```
<hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
```

- **Device capabilities**: Supported NETCONF versions along with NETCONF base capabilities, NETCONF optional features, and YANG modules implemented on IOS XE. (The comments are not included in the actual output.)

```
<capabilities>
  <!-- NETCONF base versions -->
  <capability>urn:ietf:params:netconf:base:1.0</capability>
  <capability>urn:ietf:params:netconf:base:1.1</capability>
  <output omitted>

  <!-- NETCONF features -->
  <capability>urn:ietf:params:netconf:capability:writable-running:1.0</capability>
  <capability>urn:ietf:params:netconf:capability:rollback-on-error:1.0</capability>
  <output omitted>

  <!-- YANG modules supported by the device -->
  <capability>http://cisco.com/ns/yang/Cisco-IOS-XE-native?revision=2024-02-
01</capability>
  <capability>http://cisco.com/ns/yang/cisco-xe-openconfig-interfaces-ext
             ?module=cisco-xe-openconfig-interfaces-ext&revision=2018-07-
14</capability>
  <output omitted>

</capabilities>
```

- **A session ID**: the unique identifier for this NETCONF session

```
<session-id>24</session-id>
```

- **The NETCONF end-of-message marker**]]>]]>

```
</hello>]]>]]>
```

This message confirms that the NETCONF service is running properly and is ready to receive RPC requests from your client.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Step 2: Start a NETCONF session by sending a Client <hello> message.

a. After D1 sends its server **<hello>** message, the NETCONF protocol requires the client to respond with its own **<hello>**. This completes the NETCONF session setup and tells the device which NETCONF version the client supports.

Paste the following **<hello>** message into your NETCONF SSH session:

```
<hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
 <capabilities>
   <capability>urn:ietf:params:netconf:base:1.0</capability>
 </capabilities>
</hello>
]]>]]>
```

b. Where do these two URNs come from?

They come *directly from the server's <hello> message* that D1 sent at the beginning of the session.

1) **NETCONF XML Namespace (xmlns=…)**: This identifies the NETCONF base protocol schema used for all NETCONF messages.

From D1's (server's) <hello>:

```
<hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
```

So, the PC (client) <hello> uses:

```
xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
```

This tells the server, *"The XML elements I will be sending follow the NETCONF base 1.0 standard."*

Note: The XML namespace **urn:ietf:params:xml:ns:netconf:base:1.0** is required because it tells the device (D1) which XML schema the message uses. NETCONF relies on this namespace to uniquely identify NETCONF elements such as <hello>, <rpc>, <rpc-reply>, <get>, <get-config>, and <edit-config>. Without it, D1 would not recognize the message as valid NETCONF.

2) **NETCONF Base Capability (which version the client supports)**: The client must choose one NETCONF base version that the server advertised.

From D1's (server's) <hello>:

```
<capability>urn:ietf:params:netconf:base:1.0</capability>
```

So, the PC (client) <hello> uses:

```
<capability>urn:ietf:params:netconf:base:1.0</capability>
```

This tells the server, *"I support NETCONF Base 1.0, which you (D1) advertised as supported."*

Note: The client only needs to declare the NETCONF base capability. This selects the NETCONF version and message framing rules for the session. The client does **not** need to list every capability the server advertised—just one supported base version is enough to establish the NETCONF session.

c. On D1, use the **show netconf-yang sessions** command to verify that a NETCONF session has been successfully started. If not, you may have entered a typo in the XML <hello> message above.

```
D1# show netconf-yang sessions
R: Global-lock on running datastore
C: Global-lock on candidate datastore
S: Global-lock on startup datastore

Number of sessions : 1
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
session-id  transport    username             source-host            global-lock
-------------------------------------------------------------------------------
24          netconf-ssh  cisco                10.23.0.10             None
```

This output shows information about all active NETCONF sessions on the device. In this example:

- **Number of sessions: 1**: There is one active NETCONF session.
- **session-id 24**: The unique ID assigned to your session (this ID is also shown in the device's <hello> message).
- **transport: netconf-ssh**: Indicates the session is running over SSH on TCP port 830.
- **username: cisco**: The authenticated user for the NETCONF session.
- **source-host: 10.23.0.10**: The IP address of the PC initiating the NETCONF session.
- **global-lock: None**: No configuration datastore is locked by this session (locking is optional and used for advanced NETCONF operations).

The lines at the top:

```
R: Global-lock on running datastore
C: Global-lock on candidate datastore
S: Global-lock on startup datastore
```

This shows the lock status of each configuration datastore. In this lab, you are not using datastore locking, so all locks should remain **None**. You will learn more about datastores later in the lab.

This confirms that your NETCONF session is live and ready to send RPC messages.

Step 3: Send NETCONF RPC Messages to D1.

After the client and server have exchanged their **<hello>** messages, the NETCONF session is fully established. At this point, the device is ready to receive Remote Procedure Calls (RPCs) from the client. An RPC is an XML message that requests an operation from the device, such as retrieving configuration, reading interface state, or modifying the running configuration.

Every NETCONF operation is sent inside an **<rpc>** element, and the device responds with an **<rpc-reply>** message that contains the requested data or the result of the operation. Now that the session is active, you can begin sending RPCs to D1.

During an SSH-based NETCONF session, the client can send RPCs for many types of operations. The list below lists several common NETCONF operations you will encounter:

- **<get>**: Retrieves operational (state) data and parts of the running configuration.
- **<get-config>**: Retrieves all or part of a specific configuration datastore.
- **<edit-config>**: Modifies all or part of a configuration datastore.
- **<copy-config>**: Replaces an entire configuration datastore with another.
- **<delete-config>**: Deletes a configuration datastore.
- **<commit>**: Copies the candidate datastore into the running datastore.
- **<lock> / <unlock>**: Locks or unlocks a configuration datastore to prevent conflicts.
- **<close-session>**: Gracefully closes the NETCONF session.
- **<kill-session>**: Forcefully terminates a NETCONF session.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Note: Several of these operations (such as **<lock>**, **<commit>**, and **<candidate>**) relate to NETCONF's datastore architecture. You will learn more about datastores later in the lab when you run your first **<get-config>** operation.

a. To retrieve interface information from D1, paste the following NETCONF RPC into your SSH NETCONF session. This RPC uses the **<get>** operation along with a YANG-based **<filter>** that selects the **ietf-interfaces** model.

```
<rpc message-id="103" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
 <get>
  <filter>
    <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
   </filter>
 </get>
</rpc>
```

What does this RPC do?

- **<rpc>**: Wraps the request.
- **message-id="103"**: A client-chosen identifier to match request/response pairs.
- **<get>**: Requests operational data and parts of running configuration.
- **<filter>**: Limits the reply to the **YANG interface model** only.
- **ietf-interfaces**: The standard YANG model that describes interface state and configuration.

This is the NETCONF equivalent of, *"Show me all interfaces using the standardized YANG model."*

In NETCONF, the XML tags you include inside your **<filter>** must match the structure and names of the YANG model used by the device. **<interfaces>** refers to the **ietf-interfaces** model (configuration data).

When retrieving *all* interfaces using NETCONF, the **<filter>** can use either a self-closing tag or a full open/close tag, and the device treats them exactly the same. For example, the following two statements are equivalent:

```
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
```

and

```
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"></interfaces>
```

b. D1 responds with an **<rpc-reply>** message containing XML data for *all* interfaces. Because XML does not require indentation or newlines, the output is returned as one long continuous line.

For example:

```
<?xml version="1.0" encoding="UTF-8"?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-
id="103"><data><interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-
interfaces"><interface><name>GigabitEthernet0/0</name><type
xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type><enabled>true</enabled><ipv4
xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"></ipv6></interface><interface><name>GigabitEthernet1/0/1</name><type
xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type><enabled>false</enabled><ipv4
xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"></ipv6></interface><interface><name>GigabitEthernet1/0/1</name><description>Link
to Sales network</description><type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type><enabled>true</enabled><ipv4
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>10.11.0.1</ip><netmask>255.255.0.0</netmask></address></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>2001:db8:acad:11::1</ip><prefix-length>64</prefix-
length></address></ipv6></interface><interface><name>GigabitEthernet1/0/2</name><descr
iption>Link to Engineering network</description><type
xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type><enabled>true</enabled><ipv4
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>10.12.0.1</ip><netmask>255.255.0.0</netmask></address></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>2001:db8:acad:12::1</ip><prefix-length>64</prefix-
length></address></ipv6></interface>


<output omitted>


<interface><name>GigabitEthernet1/0/23</name><description>Link to
PC</description><type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type><enabled>true</enabled><ipv4
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>10.23.0.1</ip><netmask>255.255.0.0</netmask></address></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>2001:db8:acad:23::1</ip><prefix-length>64</prefix-
length></address></ipv6></interface><interface>


<output omitted>


<interface><name>GigabitEthernet1/1/4</name><type
xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type><enabled>true</enabled><ipv4
xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"></ipv6></interface><interface><name>Vlan1</name><type
xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:l3ipvlan</type><enabled>true</enabled><ipv4
xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"></ipv6></interface></interfaces></data></rpc-reply>]]>]]>
```

d. Select and copy the XML that was returned, but do not include the NETCONF end marker, "**]]>]]>**". This sequence is the NETCONF *end-of-message* marker and is **not part of the XML** itself.

e. Because devices return XML in a single long line, use any web-based "XML Prettify" or "XML Formatter" tool to make the output easier to read. Just copy-paste the unformatted XML into the web-based formatter. After formatting, the XML should resemble the following (content shortened for clarity):

```
<?xml version="1.0" encoding="UTF-8"?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="103">
  <data>
    <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
      <interface>
        <name>GigabitEthernet0/0</name>
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type>
        <enabled>true</enabled>
        <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
        <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
      </interface>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
      <interface>
        <name>GigabitEthernet1/0/1</name>
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type>
        <enabled>false</enabled>
        <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
        <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
      </interface>
      <interface>
        <name>GigabitEthernet1/0/1</name>
        <description>Link to Sales network</description>
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type>
        <enabled>true</enabled>
        <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
          <address>
            <ip>10.11.0.1</ip>
            <netmask>255.255.0.0</netmask>
          </address>
        </ipv4>
        <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
          <address>
            <ip>2001:db8:acad:11::1</ip>
            <prefix-length>64</prefix-length>
          </address>
        </ipv6>
      </interface>
<output omitted>
<interface>
        <name>GigabitEthernet1/0/23</name>
        <description>Link to PC</description>
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type>
        <enabled>true</enabled>
        <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
          <address>
            <ip>10.23.0.1</ip>
            <netmask>255.255.0.0</netmask>
          </address>
        </ipv4>
        <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
          <address>
            <ip>2001:db8:acad:23::1</ip>
            <prefix-length>64</prefix-length>
          </address>
        </ipv6>
      </interface>
      <interface>
        <name>Vlan1</name>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:l3ipvlan</type>
        <enabled>true</enabled>
        <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
        <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
      </interface>
    </interfaces>
  </data>
</rpc-reply>
```

This RPC returns **all interface data** from the YANG model **ietf-interfaces**.

Specifically, it displays:

- All physical interfaces (GigabitEthernet1/0/x, 1/1/x, etc.)
- Any routed interfaces or switched virtual interfaces (e.g., Vlan1)
- Interface descriptions
- Whether the interface is enabled/disabled
- IPv4 and IPv6 addresses
- Interface type (mapped to IANA if-types)

In short, this RPC is the NETCONF equivalent of, *"Show me every interface, including its operational state and IP addressing, in structured YANG format."*

RESTCONF Equivalent: For those who have already done the RESTCONF lab, recall that the following RESTCONF GET returns the same interface data:

```
GET https://10.23.0.1/restconf/data/ietf-interfaces:interfaces
```

Step 4: Close the NETCONF session.

a. To end the NETCONF session, the client must send an RPC containing the **<close-session>** operation. Paste the following RPC into your NETCONF SSH session:

```
<rpc message-id="9999999" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
 <close-session />
</rpc>
```

Information in this RPC call includes:

- **message-id="9999999"**: This is a client-chosen identifier used to match the request with the device's <rpc-reply>. Any unique value is acceptable.
- **xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"**: This namespace identifies the message as part of the NETCONF base protocol.

b. After sending the request, the NETCONF session closes and you are returned to your terminal prompt. You can verify that the session has ended by returning to the device and entering:

```
D1# show netconf-yang sessions
There are no active sessions
```

The session previously listed should no longer appear.

Part 4: Using NETCONF Interactive Mode to Retrieve Interface-State Information

In this part of the lab, you will use NETCONF interactive mode to retrieve both:

1. Interface configuration and addressing information, using the **ietf-interfaces** YANG model, and

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

2. Operational/state information, using the **ietf-interfaces-state** model.

We will be retrieving this information for a specific interface (GigabitEthernet1/0/23).

Step 1: Establish an SSH NETCONF connection.

a. Before sending RPCs, remember to SSH into the device using the NETCONF parameters:

```
ssh -oHostKeyAlgorithms=+ssh-dss cisco@10.23.0.1 -p 830 -s netconf
```

b. Send your client **<hello>** message to complete NETCONF session setup:

```
<hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <capabilities>
    <capability>urn:ietf:params:netconf:base:1.0</capability>
  </capabilities>
</hello>
]]>]]>
```

There won't be any output. But on D1, verify the session is active using the **show netconf-yang sessions** command. When the session is active, you can begin sending NETCONF RPCs.

Step 2: Retrieve Configuration Information for a Specific Interface (GigabitEthernet1/0/23)

This RPC uses **ietf-interfaces** that we used previously, that contains interface configuration (name, description, IPv4/IPv6 addresses, etc.), but for a specific interface GigabithEthernet1/0/23.

a. Paste the following RPC into your NETCONF interactive session:

```
<rpc message-id="103" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <get>
    <filter>
      <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
        <interface>
          <name>GigabitEthernet1/0/23</name>
        </interface>
      </interfaces>
    </filter>
  </get>
</rpc>
]]>]]>
```

In short, this is the NETCONF/YANG equivalent of, "Show me the interface information for GigabitEthernet1/0/23."

Previously, you retrieved information for *all interfaces* by sending an empty **<interfaces>** element in the filter. When requesting configuration for only a **specific** interface, the filter must include an **<interface>** element with a **<name>** that matches the interface you want, such as GigabitEthernet1/0/23. This tells NETCONF to return only that interface rather than the entire interface list, which makes the output smaller and easier to analyze.

The basic idea is an empty **<interfaces>** returns everything, while adding **<interface><name>**…**</name></interface>** returns only the matching entry.

Note: If you want to explore additional filtering examples, Cisco's YANG Explorer (developer.cisco.com) or public YANG model browsers provide helpful reference examples.

D1 responds with the following output:

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
<?xml version="1.0" encoding="UTF-8"?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-
id="103"><data><interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-
interfaces"><interface><name>GigabitEthernet1/0/23</name><description>Link to
PC</description><type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type><enabled>true</enabled><ipv4
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>10.23.0.1</ip><netmask>255.255.0.0</netmask></address></ipv4><ipv6
xmlns="urn:ietf:params:xml:ns:yang:ietf-
ip"><address><ip>2001:db8:acad:23::1</ip><prefix-length>64</prefix-
length></address></ipv6></interface></interfaces></data></rpc-reply>]]>]]>
```

b. After removing the NETCONF end-marker **]]>]]>**, using an XML formatter, the output is:

```
<?xml version="1.0" encoding="UTF-8"?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="103">
  <data>
    <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
      <interface>
        <name>GigabitEthernet1/0/23</name>
        <description>Link to PC</description>
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type>
        <enabled>true</enabled>
        <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
          <address>
            <ip>10.23.0.1</ip>
            <netmask>255.255.0.0</netmask>
          </address>
        </ipv4>
        <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
          <address>
            <ip>2001:db8:acad:23::1</ip>
            <prefix-length>64</prefix-length>
          </address>
        </ipv6>
      </interface>
    </interfaces>
  </data>
</rpc-reply>
```

This is the NETCONF/YANG equivalent of the Cisco IOS command **show running-config interface GigabitEthernet1/0/23**.

RESTCONF Equivalent: For those familiar with the RESTCONF lab, this is the equivalent in RESTCONF to:

```
GET https://10.23.0.1/restconf/data/ietf-
interfaces:interfaces/interface=GigabitEthernet1%2F0%2F23
```

Step 3: Retrieve Operational State for the Same Interface

a. Using the NETCONF/YANG model **ietf-interfaces-state**, retrieve operational state information for GigabitEthernet 1/0/23.

b. Paste the following RPC into your NETCONF interactive session:

```
<rpc message-id="104" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
  <get>
    <filter>
      <interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
        <interface>
          <name>GigabitEthernet1/0/23</name>
        </interface>
      </interfaces-state>
    </filter>
  </get>
</rpc>
]]>]]>
```

Remember, in NETCONF, the XML tags you include inside your **<filter>** must match the structure and names of the YANG model used by the device. **<interfaces-state>** refers to the **ietf-interfaces-state** model (operational/state data).

c. This RPC returns dynamic information.

```
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-
id="104"><data><interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-
interfaces"><interface><name>GigabitEthernet1/0/23</name><type
xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type><admin-status>up</admin-status><oper-
status>up</oper-status><last-change>2025-11-24T01:02:49.096+00:00</last-change><if-
index>31</if-index><phys-address>f8:e9:4f:9b:7f:e7</phys-
address><speed>100000000</speed><statistics><discontinuity-time>2025-11-
23T14:49:01+00:00</discontinuity-time><in-octets>1377744</in-octets><in-unicast-
pkts>12495</in-unicast-pkts><in-broadcast-pkts>1290</in-broadcast-pkts><in-multicast-
pkts>1086</in-multicast-pkts><in-discards>0</in-discards><in-errors>0</in-errors><in-
unknown-protos>0</in-unknown-protos><out-octets>5345645</out-octets><out-unicast-
pkts>7525</out-unicast-pkts><out-broadcast-pkts>6581</out-broadcast-pkts><out-
multicast-pkts>27203</out-multicast-pkts><out-discards>0</out-discards><out-
errors>0</out-errors></statistics></interface></interfaces-state></data></rpc-
reply>]]>]]>
```

d. After removing the NETCONF end-marker **]]>]]>**, using an XML formatter, the output is:

```
<?xml version="1.0"?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="104">
  <data>
    <interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
      <interface>
        <name>GigabitEthernet1/0/23</name>
        <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-if-
type">ianaift:ethernetCsmacd</type>
        <admin-status>up</admin-status>
        <oper-status>up</oper-status>
        <last-change>2025-11-24T01:02:49.096+00:00</last-change>
        <if-index>31</if-index>
        <phys-address>f8:e9:4f:9b:7f:e7</phys-address>
        <speed>100000000</speed>
        <statistics>
          <discontinuity-time>2025-11-23T14:49:01+00:00</discontinuity-time>
          <in-octets>1377744</in-octets>
          <in-unicast-pkts>12495</in-unicast-pkts>
          <in-broadcast-pkts>1290</in-broadcast-pkts>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
            <in-multicast-pkts>1086</in-multicast-pkts>
            <in-discards>0</in-discards>
            <in-errors>0</in-errors>
            <in-unknown-protos>0</in-unknown-protos>
            <out-octets>5345645</out-octets>
            <out-unicast-pkts>7525</out-unicast-pkts>
            <out-broadcast-pkts>6581</out-broadcast-pkts>
            <out-multicast-pkts>27203</out-multicast-pkts>
            <out-discards>0</out-discards>
            <out-errors>0</out-errors>
          </statistics>
        </interface>
      </interfaces-state>
    </data>
</rpc-reply>
```

This RPC returns dynamic information such as:

- admin-status
- oper-status
- last-change timestamp
- statistics (octets, packets, errors)
- MAC address
- speed

RESTCONF Equivalent - For those familiar with the RESTCONF lab, this is the equivalent in RESTCONF to:

```
GET https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-
state/interface=GigabitEthernet1%2F0%2F23
```

e. End the NETCONF session by sending an RPC containing the **<close-session>** operation. Paste the following RPC into your NETCONF SSH session:

```
<rpc message-id="9999999" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
 <close-session />
</rpc>
```

Part 5: Using NETCONF with Python

Python provides a much easier way to work with NETCONF than interactive mode. The **ncclient** library automatically handles the NETCONF session, message framing, XML parsing, and pretty-printing — allowing you to focus on the YANG data rather than typing raw XML.

Before running any Python code, verify that the **ncclient** module is installed.

Step 1: Install the ncclient Python module (if needed)

The ncclient package is a Python library that allows your program to connect to a NETCONF device, send RPCs, and receive structured XML replies.

Install ncclient (if it is missing) using:

```
pip3 install ncclient
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Step 2: Verify that ncclient is installed

Confirm installation by listing all installed Python modules:

```
pip3 list --format=columns
```

You should see output similar to:

```
Package               Version
--------------------- ------------------
asn1crypto            0.24.0
ncclient              0.6.7
netifaces             0.10.4
netmiko               3.0.0
paramiko              2.7.1
requests              2.22.0
xmltodict             0.12.0
(output omitted)
```

Step 3: Create the Python program to display supported NETCONF capabilities.

a. Open IDLE, VS Code or another Python IDE.

b. Create a new Python file and save the file as **netconf-capabilities.py**.

c. Enter the following Python code:

```
# ========== PRINT CAPABILITIES ==========
from ncclient import manager

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

print("#Supported Capabilities (YANG models):")
for capability in m.server_capabilities:
    print(capability)

m.close_session()
```

Explanations:

```
from ncclient import manager
```

This imports the manager object from the **ncclient** library. The manager object is the main interface that **ncclient** uses to:

- Open NETCONF sessions
- Send RPCs
- Retrieve configuration and operational data
- Close sessions

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

Without this import, Python would not know how to connect to a NETCONF device.

```
m = manager.connect()
```

This function opens a NETCONF session to the IOS XE device.

- **host** – The IP address of D1
- **port=830** – The standard NETCONF-over-SSH port
- **username** / **password** – device credentials
- **hostkey_verify=False** – allows ncclient to connect without verifying SSH host keys (useful for labs)

If the connection succeeds, **ncclient** returns a **manager session object** and assigns it to the variable **m**. Using the variable name **m** keeps the code clean and short, since many operations use **m**.get(), **m**.edit_config(), **m**.get_config(), etc. You can name it anything (e.g., session), but **m** is the standard.

```
for capability in m.server_capabilities:
    print(capability)
```

Just as you saw earlier in interactive NETCONF mode, the device (D1) advertises its supported capabilities in its initial **<hello>** message. The **m.server_capabilities** attribute lists every NETCONF and YANG capability the device advertised in its initial **<hello>** message. Because **m** is our NETCONF session object, this attribute lets the Python program display the same capabilities we saw earlier in interactive mode showing which YANG models the device supports and which namespace URIs you can use in your filters.

This Python loop prints the exact same list of capabilities—but now through **ncclient** instead of raw XML. These capabilities include the NETCONF base versions (1.0 and 1.1), writable-running, candidate support, notifications, and both IETF and Cisco native YANG models (such as *ietf-interfaces* and *Cisco-IOS-XE-native*).

```
m.close_session()
```

This sends a **<close-session>** RPC to the router and ends the NETCONF SSH session cleanly.

d. Run the code and examine the output.

Note: It is helpful to pipe the output to more so you can see one screen at a time. For example, in Windows command line it would look like this:

```
C:\Users\Student> python netconf-capabilities.py | more
```

Escape at any time using **Ctrl+C**.

The output contains the same capability information you saw earlier in interactive NETCONF mode. The device's (D1) NETCONF base versions, supported features, and all available YANG modules are the same ones you saw earlier in interactive mode. The difference is only in how they are displayed.

In interactive mode, these capabilities appear as part of a large XML **<hello>** message, each wrapped in **<capability>...</capability>** tags. When using Python's **ncclient** library, those same capabilities are parsed and printed **without** the opening and closing **<capability>** XML tags, appearing instead as simple, clean text lines that are much easier to read.

```
#Supported Capabilities (YANG models):
urn:ietf:params:netconf:base:1.0
urn:ietf:params:netconf:base:1.1
urn:ietf:params:netconf:capability:writable-running:1.0
urn:ietf:params:netconf:capability:xpath:1.0
urn:ietf:params:netconf:capability:validate:1.0
urn:ietf:params:netconf:capability:validate:1.1
urn:ietf:params:netconf:capability:rollback-on-error:1.0
urn:ietf:params:netconf:capability:notification:1.0
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
 Urn:ietf:params:netconf:capability:interleave:1.0
 <output omitted>
urn:ietf:params:netconf:capability:writable-running:1.0
urn:ietf:params:netconf:capability:rollback-on-error:1.0
<output omitted>
http://cisco.com/ns/yang/Cisco-IOS-XE-native?revision=2024-02-01
http://cisco.com/ns/yang/cisco-xe-openconfig-interfaces-ext
           ?module=cisco-xe-openconfig-interfaces-ext&revision=2018-07-14
<output omitted>
```

Part 6: Using Python and NETCONF to Retrieve Device Configuration

In this part of the lab, you will use Python's **ncclient** library to connect to D1's NETCONF server and retrieve configuration data. Just as you did in interactive NETCONF mode, Python can issue NETCONF RPCs and receive XML replies but with the advantage of automation, formatting, and the ability to filter specific YANG models.

Step 1: Retrieve the Entire Running Configuration (Unfiltered).

a. Start by creating a Python program that retrieves the *entire* running configuration from D1.

Create a new file named **netconf-running.py** and enter the following code:

```
# ========== PRINT RUNNING CONFIGURATION ==========
from ncclient import manager
import xml.dom.minidom

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

netconf_reply = m.get_config(source="running")
print("\nRunning without filter:")
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

print("\nRunning with Cisco-IOS-XE-native filter:")
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

```
import xml.dom.minidom
```

This module is used to "prettify" XML output by inserting indentation and line breaks.
In this program we don't prettify anything yet, but we include it because many NETCONF examples later will use it.

```
netconf_reply = m.get_config(source="running")
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

This line sends a NETCONF **<get-config>** RPC to the device using the existing session (**m**).

The parameter **source="running"** tells the router to return the contents of the **running datastore**—the current active configuration. The device sends back an XML **<rpc-reply>**, and **ncclient** stores it in the variable **netconf_reply**.

Note: The **"running" datastore** is the device's active configuration—equivalent to what you see in IOS with **show running-config**. NETCONF allows you to retrieve this datastore directly with **get_config(source="running")**.

```
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())
```

The **.xml** attribute extracts the raw XML text from **netconf_reply**. **xml.dom.minidom.parseString()** loads that XML into Python's XML parser, and .**toprettyxml()** formats it with indentation and line breaks. This makes the XML output much easier to read, compared to the unformatted single-line XML returned by NETCONF.

b. Run the code and examine the output. It will take a few seconds to run.

```
Running without filter:
<?xml version="1.0" ?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:11adced4-c2b0-
4203-ac44-4e65c49b0cd2">
        <data>
                <cloud-services-cfg-data xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-
cloud-services-cfg">
                        <cloudm-config/>
                </cloud-services-cfg-data>
                <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">

                        <version>17.12</version>
                        <memory>
                                <free>
                                        <low-watermark>
                                                <processor>130178</processor>
                                        </low-watermark>
                                </free>
                        </memory>
                        <service>
                                <timestamps>
                                        <debug-config>
                                                <datetime>
                                                        <msec/>
                                                </datetime>
                                        </debug-config>
                                        <log-config>
                                                <datetime>
                                                        <msec/>
                                                </datetime>
                                        </log-config>
                                </timestamps>
                                <dhcp/>

                        </service>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                        <platform>
                                <punt-keepalive xmlns="http://cisco.com/ns/yang/Cisco-
IOS-XE-platform">
                                        <disable-kernel-core>true</disable-kernel-
core>
                                </punt-keepalive>
                        </platform>
                        <hostname>D1</hostname>
                        <enable>
                                <secret>
                                        <type>9</type>

<secret>$9$cOSg3Vu5Xjhx7.$ue4dJCTZScWiTlkvvPDCsGMHmKDdO968K9g5OYOr6G6</secret>
                                </secret>
                        </enable>
                        <username>
                                <name>cisco</name>
                                <privilege>15</privilege>
                                <password>
                                        <encryption>0</encryption>
                                        <password>cisco123!</password>

                                </password>
                        </username>
<output omitted>
```

Why does this produce so much output?

The NETCONF **<get-config>** operation retrieves everything stored in the **running datastore**, which includes:

- Interface configurations
- VLANs
- Routing
- QoS
- System services
- Platform-specific Cisco IOS XE modules
- Dozens of other YANG-modeled components

A full IOS XE running configuration using NETCONF is far larger than the familiar CLI **show running-config.** NETCONF returns all YANG-modeled data, not just CLI text, which can easily exceed thousands of lines of XML.

For this reason, retrieving the full running configuration is rarely useful in its raw form, especially for beginners.

The running datastore

When no filter is used as in, **netconf_reply = m.get_config(source="running")**, **ncclient** asks the device to return the entire **running datastore**, with *all* YANG models included. Because the running datastore contains many different models (Cisco native, IETF standard models, OpenConfig models, feature-specific modules, operational containers, and more), the response is very large and includes every piece of configuration that NETCONF manages.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

This is why the unfiltered output is far larger than the filtered native-model version. It contains every model in the running datastore, not just the Cisco IOS XE native configuration.

Step 2: Retrieve the Running Configuration Using a Native YANG Filter.

a. To make NETCONF output manageable, you can filter the response to only include the YANG model you want. Create a new file named **netconf-running-filtered.py** and enter:

```
# ==== PRINT RUNNING CONFIGURATION WITH CISCO-IOS-XE-NATIVE FILTER ===
from ncclient import manager
import xml.dom.minidom

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

netconf_filter = """
<native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native"/>
"""

netconf_reply = m.get_config(
    source="running",
    filter=("subtree", netconf_filter)
)

print("\nRunning with Cisco-IOS-XE-native filter:")
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

```
netconf_filter = """
<native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native"/>
"""
```

This creates an XML filter that tells NETCONF to return **only** the parts of the running configuration defined by the **Cisco-IOS-XE-native** YANG model. Because the **<native>** element is written as a **self-closing tag**, it requests *all data* inside that model without specifying individual fields.

```
netconf_reply = m.get_config(
    source="running",
    filter=("subtree", netconf_filter)
)
```

This statement performs a **get_config()** operation on the running datastore, but now includes the filter. The filter ("subtree", **netconf_filter**) argument means:

- **source="running"** - Tells the router to return the contents of the **running datastore**

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

- **subtree** - Use XML filtering based on the structure of the YANG model
- **netconf_filter** - the XML defining *which* YANG model or elements to return. In this case, the device (D1) receives the request, **"***Return only the subtree of configuration defined inside the <native> YANG model.***"**

The device (D1) sends back an XML **<rpc-reply>** containing what is closer to a Cisco IOS running-config file and **ncclient** stores it in the variable **netconf_reply**.

This dramatically reduces the amount of returned XML, showing only the device's native IOS XE configuration instead of every YANG model on the system.

Note: NETCONF supports different filtering styles, but **subtree filters** are the simplest and most commonly used in CCNA/CCNP-level automation. A subtree filter uses normal XML to specify exactly which YANG model or element you want returned. Because the filter structure matches the YANG model's hierarchy, it is easy to read, easy to write, and works consistently across all IOS XE devices. For this reason, all filters in this lab use the subtree format.

b. Run the code and examine the output.

```
Running with Cisco-IOS-XE-native filter:
<?xml version="1.0" ?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:6642ddb4-ea28-
458a-855a-3e89dcda2901">
        <data>
                <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
                        <version>17.12</version>
                        <memory>
                                <free>
                                        <low-watermark>
                                                <processor>130178</processor>
                                        </low-watermark>
                                </free>
                        </memory>
                        <service>
                                <timestamps>
                                        <debug-config>
                                                <datetime>
                                                        <msec/>
                                                </datetime>
                                        </debug-config>
                                        <log-config>
                                                <datetime>
                                                        <msec/>
                                                </datetime>
                                        </log-config>
                                </timestamps>
                                <dhcp/>
                        </service>
                        <platform>
                                <punt-keepalive xmlns="http://cisco.com/ns/yang/Cisco-
IOS-XE-platform">
                                        <disable-kernel-core>true</disable-kernel-
core>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                        </punt-keepalive>
                    </platform>
                    <hostname>D1</hostname>
                    <enable>
                        <secret>
                            <type>9</type>

<secret>$9$cOSg3Vu5Xjhx7.$ue4dJCTZScWiTlkvvPDCsGMHmKDdO968K9g5OYOr6G6</secret>
                        </secret>
                    </enable>
                    <username>
                        <name>cisco</name>
                        <privilege>15</privilege>
                        <password>
                            <encryption>0</encryption>
                            <password>cisco123!</password>
                        </password>
                    </username>
<output omitted>
```

Why is there less output this time?

The filter **<native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native"/>** tells the device, "**Return only the data defined in the Cisco IOS XE native YANG model**."

This **excludes**:

- All IETF models
- Operational state data
- OpenConfig models
- Platform-specific extensions
- Any other Cisco YANG models not part of the native schema

Because the *native* model represents configuration in a way that closely matches how IOS XE organizes its CLI, this filtered output is much closer to what you would normally see when using the **show running-config** command.

The running datastore

This filter is applied to the **running datastore**, which is the live, active configuration of the device. By combining: **source="running"** and the **<native .../>** filter. This returns only the portion defined in the Cisco native YANG model. Therefore, you get a focused view of the **running datastore** expressed through the native model.

In other words: You are still retrieving the running configuration but only the parts represented in the Cisco IOS XE native YANG schema, not the full set of models contained in the datastore.

Part 7: Using Python and NETCONF to Retrieve Interface Information

In this section, you will use Python together with **ncclient** to retrieve interface data from D1 using the standard IETF YANG models. Python allows you to automate the same operations—sending **<get-config>** and **<get>** RPCs while applying YANG filters to retrieve only the information you need.

You will retrieve both **configuration data** (using the **ietf-interfaces** model) and **operational/state data** (using the **interfaces-state** model), first for *all* interfaces and then for a *single* interface (GigabitEthernet1/0/23). These examples will show how NETCONF filters allow you to precisely target the

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

data you want, avoiding the large and unreadable output that comes from retrieving the entire running datastore.

Step 1: Retrieve Interface Configuration Information for All Interfaces.

a. Create a new Python program that retrieves the configuration information for *all* interfaces from D1.

Create a new file named **netconf-interfaces.py** and enter the following code:

```
# ==== PRINT CONFIGURATION INFORMATION FOR ALL INTERFACES ====
from ncclient import manager
import xml.dom.minidom

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

netconf_filter = """
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
"""

print("\n# get-config: Interface configuration information for ALL
interfaces")
netconf_reply = m.get_config(
    source="running",
    filter=("subtree", netconf_filter)
)
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

```
netconf_filter = """
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
"""
```

The **netconf_filter** variable contains an XML filter that selects the **ietf-interfaces** YANG model, which defines the configuration data for all interfaces on the device. This filter tells NETCONF, "*Return only the configuration data defined in the ietf-interfaces model*."

Once again, in NETCONF, the XML tags you include inside your **<filter>** must match the structure and names of the YANG model used by the device. **<interfaces>** refers to the **ietf-interfaces** model (configuration data).

```
netconf_reply = m.get_config(
    source="running",
    filter=("subtree", netconf_filter)
)
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

The **get_config()** call retrieves the running datastore but applies the filter, **netconf_filte**r, so that only the interface configuration is returned.

As a reminder:

- **source="running"** - Tells the router to return the contents of the **running datastore**
- **subtree** - Use XML filtering based on the structure of the YANG model

The device sends back an XML **<rpc-reply>**, and **ncclient** stores it in the variable **netconf_reply**.

```
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())
```

Using the information stored in **netconf_reply** this produces a clean, structured XML output showing configuration details (names, descriptions, IPv4/IPv6 addresses, enabled state, etc.) for all interfaces, but without the rest of the running configuration.

b. Run the code and examine the output.

The NETCONF reply lists **every interface defined in the ietf-interfaces YANG model**, including all physical interfaces (GigabitEthernet), switched virtual interfaces (SVIs), and any other logical interfaces the device supports. For each interface, the output includes its name, type, enabled/disabled state, and any configured IPv4 or IPv6 addresses. Because no specific interface was filtered, the router returns the complete interface configuration for the entire device.

```
# get-config: running config for ALL interfaces
<?xml version="1.0" ?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:a3e17197-11f3-
4ad9-81ec-791bca458b7b">
        <data>
                <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
                        <interface>
                                <name>GigabitEthernet0/0</name>
                                <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type>
                                <enabled>true</enabled>
                                <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
                                <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip"/>
                        </interface>
                        <interface>
                                <name>GigabitEthernet1/0/1</name>
                                <description>Link to Sales network</description>
                                <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type>
                                <enabled>true</enabled>
                                <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
                                        <address>
                                                <ip>10.11.0.1</ip>
                                                <netmask>255.255.0.0</netmask>
                                        </address>
                                </ipv4>
                                <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
                                        <address>
                                                <ip>2001:db8:acad:11::1</ip>
                                                <prefix-length>64</prefix-length>
                                        </address>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                                </ipv6>
                        </interface>
<output omitted>
                        <interface>
                                <name>GigabitEthernet1/0/2</name>
                                <description>Link to Engineering network</description>
                                <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type>
                                <enabled>true</enabled>
                                <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
                                        <address>
                                                <ip>10.12.0.1</ip>
                                                <netmask>255.255.0.0</netmask>
                                        </address>
                                </ipv4>
                                <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
                                        <address>
                                                <ip>2001:db8:acad:12::1</ip>
                                                <prefix-length>64</prefix-length>
                                        </address>
                                </ipv6>
                        </interface>
<output omitted>
```

c. The Python code above using NETCONF produces the same result as:

- **NETCONF Interactive Mode**:

```
<rpc message-id="103" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
 <get>
  <filter>
    <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
   </filter>
 </get>
</rpc>
```

- **RESTCONF Request (for those familiar with RESTCONF)**:

  Request Type: GET

  URL: https://10.23.0.1/restconf/data/ietf-interfaces:interfaces

Step 2: Retrieve Interface Configuration Information for a Specific interfaces.

a. Create a new Python program that retrieves the configuration information for *a specific* interface (GigabitEthernet 1/0/23) from D1. Create a new file named **netconf-specific-interface.py** and enter the following code:

```
# ==== PRINT CONFIGURATION INFORMATION FOR A SPECIFIC INTERFACE ====
from ncclient import manager
import xml.dom.minidom

INTERFACE_NAME = "GigabitEthernet1/0/23"
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

netconf_filter = f"""
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
    <interface>
      <name>{INTERFACE_NAME}</name>
    </interface>
</interfaces>
"""

print("\n# get-config: running config for", INTERFACE_NAME)
netconf_reply = m.get_config(
    source="running",
    filter=("subtree", netconf_filter)
)

print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

```
INTERFACE_NAME = "GigabitEthernet1/0/23"
```

The **INTERFACE_NAME** variable stores the exact interface name you want to query. Using a variable makes the program easier to reuse. If you want to test a different interface later, you only need to change this one line instead of editing the XML filter.

```
netconf_filter = f"""
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
    <interface>
      <name>{INTERFACE_NAME}</name>
    </interface>
</interfaces>
"""
```

Here, **netconf_filter** is defined as an f-string (note the leading f before the quotes). The **{INTERFACE_NAME}** placeholder is replaced by the value of the **INTERFACE_NAME** variable. The XML filter now includes an **<interface>** element with a **<name>** that matches the specific interface value from INTERFACE_NAME. In other words, this filter tells NETCONF, "*From the ietf-interfaces model, return only the interface whose name is GigabitEthernet1/0/23.*"

```
netconf_reply = m.get_config(
    source="running",
    filter=("subtree", netconf_filter)
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
)
```

As before, **m.get_config()** reads from the **running** datastore, but now it applies the more specific **netconf_filter**, so only the configuration for the single interface identified by **INTERFACE_NAME** is returned. The device sends back an XML **<rpc-reply>**, and **ncclient** stores it in the variable **netconf_reply**.

```
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())
```

Using the information stored in **netconf_reply,** the final **print(...)** line uses **xml.dom.minidom** to pretty-print the XML reply, giving you a clean, readable view of the configuration for just GigabitEthernet1/0/23 instead of every interface on the device.

b. Run the code and examine the output.

The NETCONF reply shows only the configuration data for the specific interface you requested—in this case, **GigabitEthernet1/0/23**. Because the filter targeted a single **<interface>** element within the **ietf-interfaces** YANG model, the device returns a small, focused XML response instead of the full interface list. The output includes the interface name, description, enabled/disabled state, and its IPv4 and IPv6 addresses, exactly as they exist in the running datastore.

```
# get-config: running config for GigabitEthernet1/0/23
<?xml version="1.0" ?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:9b1c87a5-a741-
4cc3-bb86-46cf57685f71">
        <data>
                <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
                        <interface>
                                <name>GigabitEthernet1/0/23</name>
                                <description>Link to PC</description>
                                <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type>
                                <enabled>true</enabled>
                                <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
                                        <address>
                                                <ip>10.23.0.1</ip>
                                                <netmask>255.255.0.0</netmask>
                                        </address>
                                </ipv4>
                                <ipv6 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
                                        <address>
                                                <ip>2001:db8:acad:23::1</ip>
                                                <prefix-length>64</prefix-length>
                                        </address>
                                </ipv6>
                        </interface>
                </interfaces>
        </data>
</rpc-reply>
```

c. The Python code above using NETCONF produces the same result as:

- **NETCONF Interactive Mode**:

```
<rpc message-id="103" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <get>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
      <filter>
        <interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
          <interface>
            <name>GigabitEthernet1/0/23</name>
          </interface>
        </interfaces>
      </filter>
    </get>
  </rpc>
  ]]>]]>
```

- **RESTCONF Request (for those familiar with RESTCONF)**:

  Request Type: GET

  URL: https://10.23.0.1/restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1%2F0%2F23

Step 3: Retrieve Interface-State Configuration Information for All Interfaces.

a. Create a new Python program that retrieves the operational state information for *all* interfaces from D1. Create a new file named **netconf-interfaces-state.py** and enter the following code:

```
# ===== PRINT OPERATIONAL/STATE DATA FOR ALL INTERFACES =====
from ncclient import manager
import xml.dom.minidom

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

netconf_filter = """
<interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
"""

print("\n# get: operational state for ALL interfaces")
netconf_reply = m.get(
    filter=("subtree", netconf_filter)
)

print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

```
netconf_filter = """
<interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"/>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

"""

The filter, **netconf_filter**, selects the ***ietf-interfaces*** **"interfaces-state**" YANG model, which provides operational (live) interface data instead of configuration information from the device's running datastore. In other words, it retrieves real-time values such as status, counters, and speed, not the settings applied in the configuration. Using this filter tells the device. "*Return the real-time status and statistics for all interfaces*."

This includes fields such as:

- admin-status (up/down)
- oper-status (up/down)
- speed
- MAC address
- input/output counters
- last-change timestamp

This is different from the earlier **<interfaces>** filter, which retrieves interface configuration information.

Again, in NETCONF, the XML tags you include inside your **<filter>** must match the structure and names of the YANG model used by the device. **<interfaces-state>** refers to the **ietf-interfaces-state** model (operational/state data).

```
netconf_reply = m.get(
    filter=("subtree", netconf_filter)
)
```

The ncclient **get()** method applies the **netconf_filter** and retrieves operational (live) data instead of configuration from the running datastore. It tells the router, "*Return the real-time values that match this filter," such as interface status, counters, and link characteristics*."

Comparisons of two **ncclient** methods:

- **m.get()** method - **<get>** RPC: Retrieves **live operational data** (interfaces-state, routing statistics, hardware info, counters)
- **m.get_config()** method - **<get-config>** RPC: Retrieves **configuration data** from the running datastore

The device sends back an XML **<rpc-reply>**, and **ncclient** stores it in the variable **netconf_reply**.

```
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())
```

Using the data in **netconf_reply**, this takes the XML returned by the device and prints it in a readable, indented format. The raw output from the router is one long line of XML, so toprettyxml() makes it much easier to read and navigate.

b. Run the code and examine the output.

The NETCONF reply shows the **operational state** for every interface on the device, as defined by the *interfaces-state* YANG model. Unlike configuration data, this output contains **live, real-time values** such as administrative/operational status, MAC address, speed, last-change timestamps, and detailed traffic counters. Each **<interface>** entry represents one physical, virtual, or logical interface currently active on the router, giving a complete snapshot of the device's operational status at the moment the **<get>** request was sent.

This information similar to what you would see in CLI commands like **show interface** but formatted using YANG-defined XML.

```
# get: operational state for ALL interfaces
<?xml version="1.0" ?>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:2750a8f0-8b4a-
4d77-8895-a08dedd6d796">
        <data>
                <interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
                        <interface>
                                <name>AppGigabitEthernet1/0/1</name>
                                <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type>
                                <admin-status>up</admin-status>
                                <oper-status>up</oper-status>
                                <last-change>2025-11-23T14:51:22.837+00:00</last-
change>
                                <if-index>37</if-index>
                                <phys-address>f8:e9:4f:9b:7f:9d</phys-address>
                                <speed>1000000000</speed>
                                <statistics>
                                        <discontinuity-time>2025-11-
23T14:49:00+00:00</discontinuity-time>
                                        <in-octets>0</in-octets>
                                        <in-unicast-pkts>0</in-unicast-pkts>
                                        <in-broadcast-pkts>0</in-broadcast-pkts>
                                        <in-multicast-pkts>0</in-multicast-pkts>
                                        <in-discards>0</in-discards>
                                        <in-errors>0</in-errors>
                                        <in-unknown-protos>0</in-unknown-protos>
                                        <out-octets>5155091</out-octets>
                                        <out-unicast-pkts>4240</out-unicast-pkts>
                                        <out-broadcast-pkts>7764</out-broadcast-pkts>
                                        <out-multicast-pkts>30798</out-multicast-pkts>
                                        <out-discards>0</out-discards>
                                        <out-errors>0</out-errors>
                                </statistics>
                        </interface>
                        <interface>
                                <name>GigabitEthernet0/0</name>
                                <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type>
                                <admin-status>up</admin-status>
                                <oper-status>lower-layer-down</oper-status>
                                <last-change>2025-11-23T14:51:14.541+00:00</last-
change>
                                <if-index>1</if-index>
                                <phys-address>f8:e9:4f:9b:7f:80</phys-address>
                                <speed>1000000000</speed>
                                <statistics>
                                        <discontinuity-time>2025-11-
23T14:49:00+00:00</discontinuity-time>
                                        <in-octets>0</in-octets>
                                        <in-unicast-pkts>0</in-unicast-pkts>
                                        <in-broadcast-pkts>0</in-broadcast-pkts>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
                                        <in-multicast-pkts>0</in-multicast-pkts>
                                        <in-discards>0</in-discards>
                                        <in-errors>0</in-errors>
                                        <in-unknown-protos>0</in-unknown-protos>
                                        <out-octets>1482</out-octets>
                                        <out-unicast-pkts>15</out-unicast-pkts>
                                        <out-broadcast-pkts>0</out-broadcast-pkts>
                                        <out-multicast-pkts>0</out-multicast-pkts>
                                        <out-discards>0</out-discards>
                                        <out-errors>0</out-errors>
                                    </statistics>
                                </interface>
<output omitted>
```

c. The Python code above using NETCONF produces the same result as:

- **NETCONF Interactive Mode**:

```
<rpc message-id="103" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <get>
    <filter>
      <interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-
interfaces"/>
    </filter>
  </get>
</rpc>
]]>]]>
```

- **RESTCONF Request (for those familiar with RESTCONF)**:

  Request Type: GET

  URL: https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-state

Step 4: Retrieve Interface-State Configuration Information for a Specific Interface.

a. Create a new Python program that retrieves the operational state information for *a specific* interface (GigabitEthernet 1/0/23) from D1. Create a new file named **netconf-specific-interface-state.py** and enter the following code:

```
# ===== PRINT OPERATIONAL/STATE DATA FOR A SPECIFIC INTERFACE =====
from ncclient import manager
import xml.dom.minidom

INTERFACE_NAME = "GigabitEthernet1/0/23"

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
netconf_filter = f"""
<interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
    <interface>
      <name>{INTERFACE_NAME}</name>
    </interface>
</interfaces-state>
"""

print("\n# get: operational state for", INTERFACE_NAME)
netconf_reply = m.get(
    filter=("subtree", netconf_filter)
)

print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

Because this code has been explained previously, here is a summary.

```
INTERFACE_NAME = "GigabitEthernet1/0/23"
```

- A Python variable that stores the interface name you want to retrieve.
- Using a variable makes it easy to change the interface by editing one line.

```
netconf_filter = f"""
<interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
    <interface>
      <name>{INTERFACE_NAME}</name>
    </interface>
</interfaces-state>
"""
```

- An XML filter written as an **f-string**, allowing {INTERFACE_NAME} to be inserted dynamically.
- Selects the *interfaces-state* model and narrows the request to one <interface> with a matching <name> element.
- Tells NETCONF: *"Return operational (live) data only for GigabitEthernet1/0/23."*

```
netconf_reply = m.get(filter=("subtree", netconf_filter))
```

- Uses the **get()** method, which retrieves **operational data**, not configuration data.
- The "subtree" filter instructs NETCONF to return only the parts of the operational datastore that match the filter.
- Stores the output to the variable **netconf_reply**.

```
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())
```

- Pretty-prints the XML returned in **<rpc-reply>** and stored in **netconf_reply**.
- Displays real-time values for the selected interface (oper-status, counters, speed, MAC address, etc.).

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

b. Run the code and examine the output.

This NETCONF reply shows the real-time operational data for **only the specified interface**, GigabitEthernet1/0/23. Because the filter selected the interfaces-state model and included a specific **<name>** element, the device returns a single <interface> entry rather than the full list.

The output includes live fields such as admin/oper status, speed, MAC address, last-change time, and detailed traffic counters. This is information similar to what you would see in CLI commands like **show interface GigabitEthernet1/0/23** but formatted using YANG-defined XML.

```
# get: operational state for GigabitEthernet1/0/23
<?xml version="1.0" ?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:5c866467-1948-
41dc-91d7-a8a2ebccbf21">
        <data>
                <interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
                        <interface>
                                <name>GigabitEthernet1/0/23</name>
                                <type xmlns:ianaift="urn:ietf:params:xml:ns:yang:iana-
if-type">ianaift:ethernetCsmacd</type>
                                <admin-status>up</admin-status>
                                <oper-status>up</oper-status>
                                <last-change>2025-11-24T01:02:48.096+00:00</last-
change>
                                <if-index>31</if-index>
                                <phys-address>f8:e9:4f:9b:7f:e7</phys-address>
                                <speed>100000000</speed>
                                <statistics>
                                        <discontinuity-time>2025-11-
23T14:49:00+00:00</discontinuity-time>
                                        <in-octets>3074126</in-octets>
                                        <in-unicast-pkts>21429</in-unicast-pkts>
                                        <in-broadcast-pkts>1424</in-broadcast-pkts>
                                        <in-multicast-pkts>1254</in-multicast-pkts>
                                        <in-discards>0</in-discards>
                                        <in-errors>0</in-errors>
                                        <in-unknown-protos>0</in-unknown-protos>
                                        <out-octets>7267377</out-octets>
                                        <out-unicast-pkts>13207</out-unicast-pkts>
                                        <out-broadcast-pkts>6589</out-broadcast-pkts>
                                        <out-multicast-pkts>27301</out-multicast-pkts>
                                        <out-discards>0</out-discards>
                                        <out-errors>0</out-errors>
                                </statistics>
                        </interface>
                </interfaces-state>
        </data>
</rpc-reply>
```

c. The Python code using NETCONF above produces the same result as:

- **NETCONF Interactive Mode**:

```
<rpc message-id="104" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
  <get>
    <filter>
      <interfaces-state xmlns="urn:ietf:params:xml:ns:yang:ietf-
interfaces">
        <interface>
          <name>GigabitEthernet1/0/23</name>
        </interface>
      </interfaces-state>
    </filter>
  </get>
</rpc>
]]>]]>
```

- **RESTCONF Request (for those familiar with RESTCONF)**:

  Request Type: GET

  URL: https://10.23.0.1/restconf/data/ietf-interfaces:interfaces-state/interface=GigabitEthernet1%2F0%2F23

Part 8: Using Python and NETCONF to Configure a Device

In this final section, you will use Python and **ncclient** not just to *retrieve* data, but to *change* the device's configuration using the NETCONF **<edit-config>** operation. Unlike **<get>** or **<get-config>,** which retrieve information from the running datastore, **<edit-config>** sends structured XML that tells the router exactly what configuration to modify.

Step 1: Sending Configuration Information to Change the Hostname

a. In this step, you will push a simple configuration change (the device hostname) to help you understand how configuration edits are sent, how the router applies them, and how NETCONF confirms the result. Create a new file named **netconf-edit-hostname.py** and enter the following code:

```
# ========== EDIT HOSTNAME ==========
from ncclient import manager
import xml.dom.minidom

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

netconf_hostname = """
<config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
     <hostname>NEWHOSTNAME</hostname>
  </native>
</config>
"""
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
netconf_reply = m.edit_config(
    target="running",
    config=netconf_hostname
)

print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

```
netconf_hostname = """
<config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
     <hostname>NEWHOSTNAME</hostname>
  </native>
</config>
"""
```

The **netconf_hostname** variable contains the XML configuration you want to apply to the device.

- All configuration edits in NETCONF must be wrapped in a top-level **<config>** element (required by the NETCONF standard).
- Inside this, the **<native>** container refers to Cisco's IOS XE native YANG model, which defines CLI-style configuration elements such as hostname, interfaces, VLANs, and routing settings.
- The line **<hostname>NEWHOSTNAME</hostname>** is the actual configuration change you are sending. This will replace the device's hostname with *NEWHOSTNAME*.

```
netconf_reply = m.edit_config(
    target="running",
    config=netconf_hostname
)
```

To apply the configuration, the program uses the ncclient **edit_config()** method:

- The **target="running"** parameter tells NETCONF to apply the change directly to the *running* datastore (the live configuration).
- The **config=netconf_hostname** parameter passes the XML payload you created.
- Together this generates a NETCONF **<edit-config>** RPC that instructs the router to modify its configuration.
- The device then sends back an **<rpc-reply>** message, which usually contains **<ok/>** if the configuration succeeded or **<rpc-error>** if it failed. **ncclient** stores this entire reply in the variable **netconf_reply**, which allows your program to print or examine the response.

Previously in this lab, when you used **get-config**, you were *retrieving* configuration data using parameters such as:

`source="running"` - Read from the running datastore

`filter=(...)` - Limit what part of the configuration is returned

With **edit-config**, you are now *writing* configuration changes instead of reading them::

`target="running"` - Apply the update to the live running configuration

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

`config=netconf_hostname` - Provide the XML payload that defines the new hostname

Because this operation modifies the configuration, no filter is used. Instead, the <config> XML payload tells NETCONF exactly what to change in the running datastore.

```
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())
```

Unlike retrieval operations—where the reply contains large amounts of data—an **<edit-config>** reply is usually short. It typically contains only an **<ok/>** element (indicating success) or an **<rpc-error>** block if something failed. Prettifying it simply makes that confirmation easier to read.

b. Run the code and examine the output.

When the program sends the **<edit-config>** request to update the hostname, the device processes the change and returns an **<rpc-reply>** message. In this case, the reply contains a simple **<ok/>,** which means the configuration update was accepted and successfully applied to the running datastore. If there had been a problem—such as invalid XML, an unsupported YANG element, or a permission issue—the reply would instead contain an **<rpc-error>** element describing the failure. For successful configuration operations, however, **<ok/>** is the standard NETCONF confirmation response.

```
<?xml version="1.0" ?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:c87f5a38-a78e-
4713-975f-f6bee5543cf3">
        <ok/>
</rpc-reply>
```

On D1, use a console or SSH connection to examine the new hostname:

```
NEWHOSTNAME#
```

c. The Python code above using NETCONF produces the same result as:

- **NETCONF Interactive Mode**:

```
<rpc message-id="200" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <edit-config>
    <target>
      <running/>
    </target>
    <config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
      <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
        <hostname>NEWHOSTNAME</hostname>
      </native>
    </config>
  </edit-config>
</rpc>
]]>]]>
```

- **RESTCONF Request (for those familiar with RESTCONF)**:

  Request Type: PATCH

  URL: https://10.23.0.1/restconf/data/Cisco-IOS-XE-native:native/hostname

  Headers (examples):

  - Content-Type: application/yang-data+json
  - Accept: application/yang-data+json

  Body:

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
        {
          "Cisco-IOS-XE-native:hostname": "NEWHOSTNAME"
        }
```

d. Change the hostname back to D1.

```
NEWHOSTNAME# conf t
NEWHOSTNAME(config)# hostname D1
D1(config)# end
D1#
```

Step 2: Sending Configuration Information to Add a Loopback Interface

a. In this step, you will use Python and NETCONF to **create a new Loopback interface** on D1. Create a new file named **netconf-add-loopback.py** and enter the following code:

```
# ========== ADD LOOPBACK 1 INTERFACE ==========
from ncclient import manager
import xml.dom.minidom

m = manager.connect(
    host="10.23.0.1",
    port=830,
    username="cisco",
    password="cisco123!",
    hostkey_verify=False
    )

netconf_loopack = """
<config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
 <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
  <interface>
   <Loopback>
    <name>1</name>
    <description>[Student Name]'s loopback</description>
    <ip>
     <address>
      <primary>
       <address>10.1.1.1</address>
       <mask>255.255.255.0</mask>
      </primary>
     </address>
    </ip>
   </Loopback>
  </interface>
 </native>
</config>
"""
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
netconf_reply = m.edit_config(
    target="running",
    config=netconf_loopack
)

print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())

m.close_session()
```

Explanations:

```
netconf_loopack = """
<config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
 <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
  <interface>
   <Loopback>
    <name>1</name>
    <description>[Student Name]'s loopback</description>
    <ip>
     <address>
      <primary>
       <address>10.1.1.1</address>
       <mask>255.255.255.0</mask>
      </primary>
     </address>
    </ip>
   </Loopback>
  </interface>
 </native>
</config>
"""
```

The **netconf_loopack** variable contains the XML payload that defines the new Loopback interface using the **Cisco IOS XE native YANG model**, which organizes configuration under the **<native>** container. Inside **<native>**, the XML follows the exact hierarchy defined by that model: **<interface>** → **<Loopback>** → **<name>**, along with a description and IPv4 address/mask. This structure mirrors how Loopback interfaces are represented in the YANG model and ensures NETCONF applies the configuration correctly.

```
netconf_reply = m.edit_config(
    target="running",
    config=netconf_loopack
)
```

The program applies this configuration using the **edit_config()** method:

- **target="running"** - tells NETCONF to write the new Loopback interface directly into the running datastore (the live configuration).
- **config=netconf_loopack** - supplies the XML payload that describes the new interface.
- Together, these produce a <edit-config> RPC that instructs the router to create Loopback1 with the specified description and IP address.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

- The device then sends back an **<rpc-reply>** message, which usually contains **<ok/>** if the configuration succeeded or **<rpc-error>** if it failed. **ncclient** stores this entire reply in the variable **netconf_reply**, which allows your program to print or examine the response.

```
print(xml.dom.minidom.parseString(netconf_reply.xml).toprettyxml())
```

This parses and prints the device's **<rpc-reply>**, contained in **netconf_reply**, which normally contains **<ok/>** if the configuration was applied successfully.

b. Run the code and examine the output.

The **<rpc-reply>** indicates that the router received and processed the **<edit-config>** request. The presence of **<ok/>** means the configuration was applied successfully with no errors.

```
<?xml version="1.0" ?>
<rpc-reply xmlns="urn:ietf:params:xml:ns:netconf:base:1.0"
xmlns:nc="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="urn:uuid:d189d491-9a95-
480e-9f7b-031055e6950e">
        <ok/>
</rpc-reply>
```

The new loopback address can be verified using the **show ip interface** command on D1. The **"other"** value the output indicates that the interface was created by a method other than the CLI, such as NETCONF, RESTCONF, or another API, rather than by a user typing commands directly in configuration mode.

```
D1# show ip interface brief | include Loopback
Loopback1              10.1.1.1        YES  other     up                    up
```

Note: If there was an error, for example Loopback1 interface already existed, you may see an RPC error message such as:

```
ncclient.operations.rpc.RPCError: inconsistent value: Device refused one or more
commands
```

c. The Python code above using NETCONF produces the same result as:

- **NETCONF Interactive Mode**:

```
<rpc message-id="300" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <edit-config>
    <target>
      <running/>
    </target>
    <config xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
      <native xmlns="http://cisco.com/ns/yang/Cisco-IOS-XE-native">
        <interface>
          <Loopback>
            <name>1</name>
            <description>[Student Name]'s loopback</description>
            <ip>
              <address>
                <primary>
                  <address>10.1.1.1</address>
                  <mask>255.255.255.0</mask>
                </primary>
              </address>
```

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

```
            </ip>
           </Loopback>
         </interface>
       </native>
     </config>
   </edit-config>
</rpc>
]]>]]>
```

- **RESTCONF Request (for those familiar with RESTCONF)**:

  Request Type: POST

  URL: https://10.23.0.1/restconf/data/Cisco-IOS-XE-native:native/interface/Loopback

  Headers (examples):

  - Content-Type: application/yang-data+json
  - Accept: application/yang-data+json

  Body:

```
{
  "Cisco-IOS-XE-native:Loopback": {
    "name": 1,
    "description": "[Student Name]'s loopback",
    "ip": {
      "address": {
        "primary": {
          "address": "10.1.1.1",
          "mask": "255.255.255.0"
        }
      }
    }
  }
}
```

Part 9: NETCONF Datastores

NETCONF organizes configuration and operational information into different **datastores**, which represent distinct versions or views of the device's configuration. Understanding these datastores helps explain how NETCONF operations such as **<get-config>**, **<edit-config>**, and **<copy-config>** behave. In this lab, you worked primarily with the **running datastore**, but it is useful to know how the other NETCONF datastores—**startup** and **candidate**—are used on NETCONF-enabled devices.

Running Datastore

The **running** datastore contains the live, active configuration currently used by the device. All changes made with **<edit-config>** in this lab were applied directly to the running datastore, meaning the configuration took effect immediately, just like editing the *running-config* in Cisco IOS CLI.

All your **get_config(source="running")** commands also read from this datastore. Because the running datastore always reflects the device's active configuration, it is the datastore most commonly used for NETCONF automation on IOS XE.

Startup Datastore

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public

The **startup** datastore contains the configuration that is loaded when the device boots, similar to *startup-config* in the Cisco CLI. Editing the running datastore does **not** automatically update the startup datastore. To make changes persistent across reboots, a NETCONF client must explicitly copy the running datastore into the startup datastore using **<copy-config>**. IOS XE devices typically support the startup datastore, and this capability is advertised in the device's **<hello>** message.

Candidate Datastore

The **candidate** datastore is an optional "staging area" for configuration changes. Instead of applying changes immediately, a NETCONF client can write changes into the candidate datastore, compare or validate them, and then use a **<commit>** operation to apply all changes at once to the running datastore. This provides transactional safety and rollback capability. The candidate datastore is used only if the device advertises the **:candidate** capability. In this lab, all edits were applied directly to the running datastore because the candidate datastore was not used.

© 2020 - 2026 Cisco and/or its affiliates. All rights reserved. Cisco Public